

FlexWATCH[®] AI SDK for Hailo-15 IP camera (Preliminary version)

Version 1.0

(May . , 20, 2026)



SEYEON TECH Co., Ltd.

<http://www.flexwatch.com>

Table of Contents

1. Background.....	3
2. AI model Integration flowchart using the FlexWATCH [®] AI SDK	5
3. How to Use Dataflow Compiler	6
A. How to create a Developer Zone account	6
B. System Requirements	8
C. Installing Dataflow Compiler.....	9
D. Verifying Installation and Introducing Key CLI Tools.....	10
E. Deep Learning and ONNX File Generation.....	10
F. Model Compilation Example (ONNX to HEF)	11
4. Hailo-15 IP Camera Detection Demo Execution Guide	14
A. Overview	14
B. Prerequisites	14
C. Network Configuration	14
5. Demo Environment Initialization	16
A. Log in to the IP Camera using SSH. (password: root).	16
B. Environment Preparation	16
C. Reboot the IP Camera.	16
D. Upload Demo Files.	16
E. Log in to IP Camera and prepare to run the demo.	17
F. Run /home/root/apps/detection/detection.sh.	17
G. View execution results.	18
6. How to Install SDK and Build Natively	19
7. Integration of .sh, HEF files on FlexWATCH [®] IP-Camera.	30
8. Appendix	31
GStreamer Pipeline Analysis	31
Troubleshooting when viewing RTSP video with VLC	34

1. Background

It supports Hailo-15H, Hailo-15M, and Hailo-15L. In this document, it is referred to as Hailo-15 .

was written to explain the purpose and approach of **the FlexWATCH® AI SDK** , which supports the more efficient and stable application of customer AI models in a FlexWATCH® IP camera environment equipped with the Hailo-15H AI accelerator .

Recently, various AI models are being rapidly developed and disseminated in the fields of video security and intelligent video analysis. However, to apply these AI technologies to actual IP camera products and ensure stable long-term operation, multiple technical factors must be considered in addition to the AI models themselves. Due to the nature of IP cameras as equipment installed and operated in the field for extended periods, continuous software maintenance in response to changes in network and security environments, support for standard protocols, expansion of streaming methods, and the maintenance of compatibility with web browsers and VMS/NVRs are essential requirements.

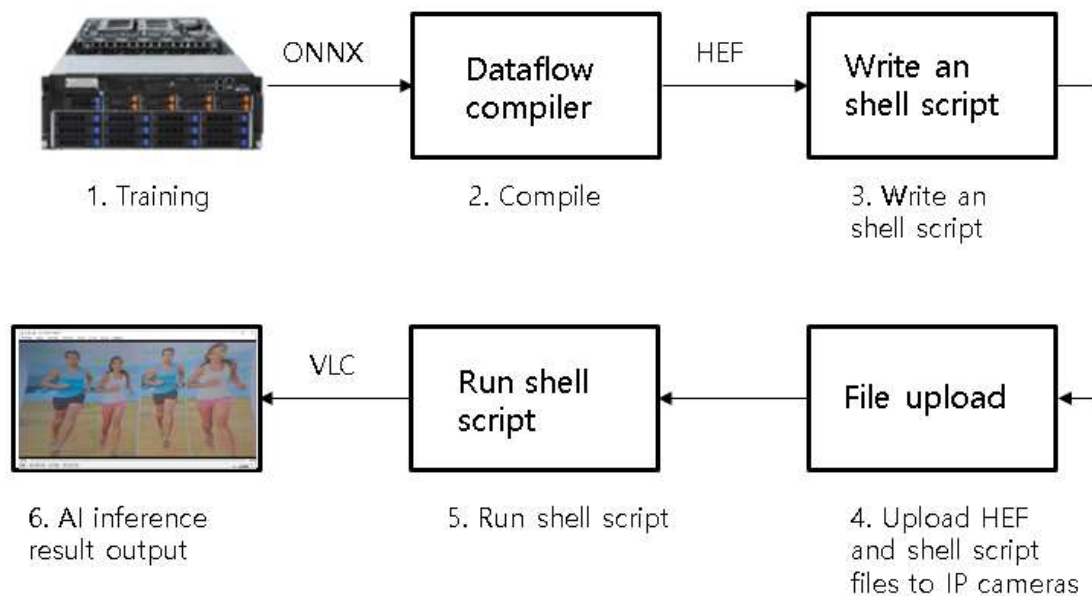
While the Hailo SDK provides powerful tools for compiling AI models and executing inference, integrating it into IP camera products requires additional elements such as video pipeline configuration, streaming processing, standard compatibility, and ensuring system stability for long-term operation. These aspects can act as a significant development burden during the commercialization phase, separate from AI model performance.

To alleviate this burden, Seyon Tech provides the FlexWATCH® IP Camera Platform based on the Hailo-15H AI Accelerator, along with the FlexWATCH® AI SDK that extends it . This SDK enables clients to focus on AI model development and performance enhancement, while providing the video processing, network streaming, standard protocols, and system integration elements required for IP cameras at the platform level.

IP cameras equipped with the FlexWATCH® AI SDK natively support various network streaming methods such as HTTP/HTTPS, RTSP/RTP, and RTSPS/SRTTP, along with ONVIF-based VMS/NVR integration, and seamlessly integrate with major web browsers and media players. This enables AI-enabled IP cameras to expand while maintaining compatibility with existing video security infrastructure.

This document provides foundational information for understanding the concepts and overall flow of the FlexWATCH® AI SDK, and subsequent chapters explain the steps for compiling, validating, running demos, and applying AI models to actual cameras.

2. AI model Integration flowchart using the FlexWATCH[®] AI SDK



The figure above is a general flowchart of the Open AI SDK supplied by our company .

First, the process involves creating a training model in a PC environment.

Subsequently, the training model developed in a PC environment uses Dataflow Compiler to target the device (FlexWATCH[®] It switches to an AI learning model running on the AI IP camera over Hailo-15 . In this document, the model is uploaded using a manual method, but In reality, You can upload through the camera's settings page. The AI model uploaded in this way can have its features configured in the web settings window, and You can also configure some inference settings. since, You can check the results of applying the AI model in a browser or control software. The next section introduces how to use the DataFlow compiler and the process of uploading and running an AI model on an IP camera.

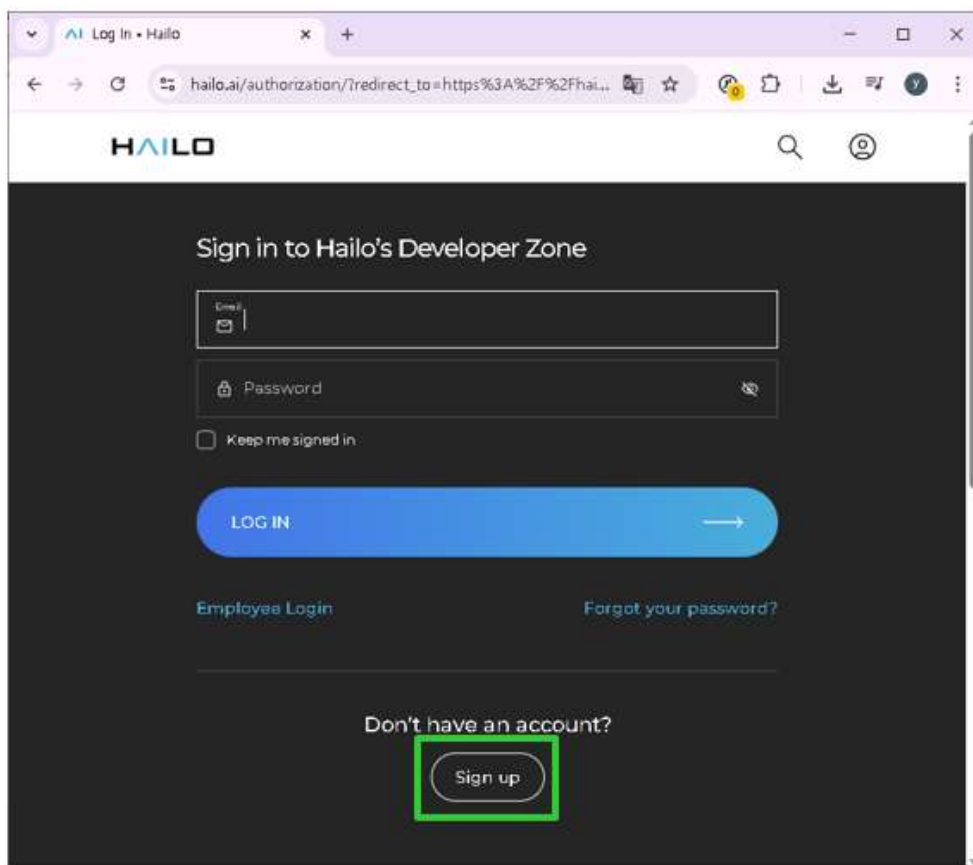
3. How to Use Dataflow Compiler

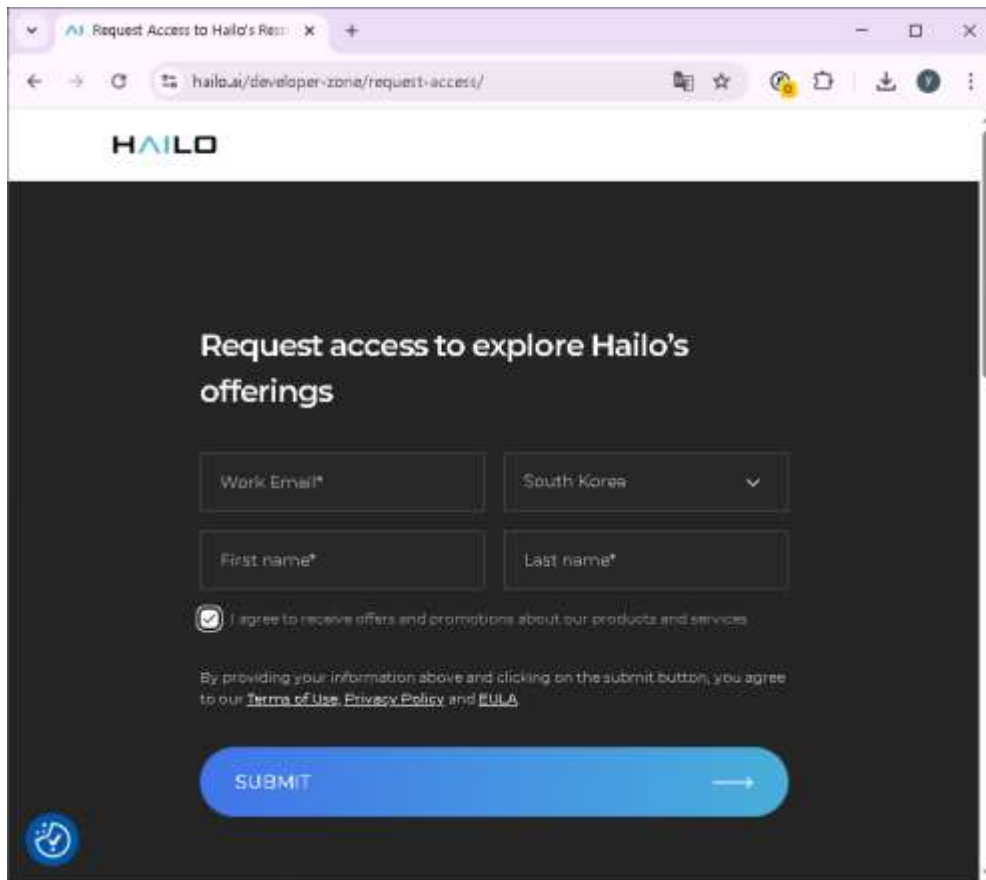
create an account in the Developer Zone and download and install the Data Flow Compiler (dfc) .

A. How to create a Developer Zone account

already have an account , you can download DFC by referring to number 5 below . The necessary documents are provided by Hailo Korea headquarters.

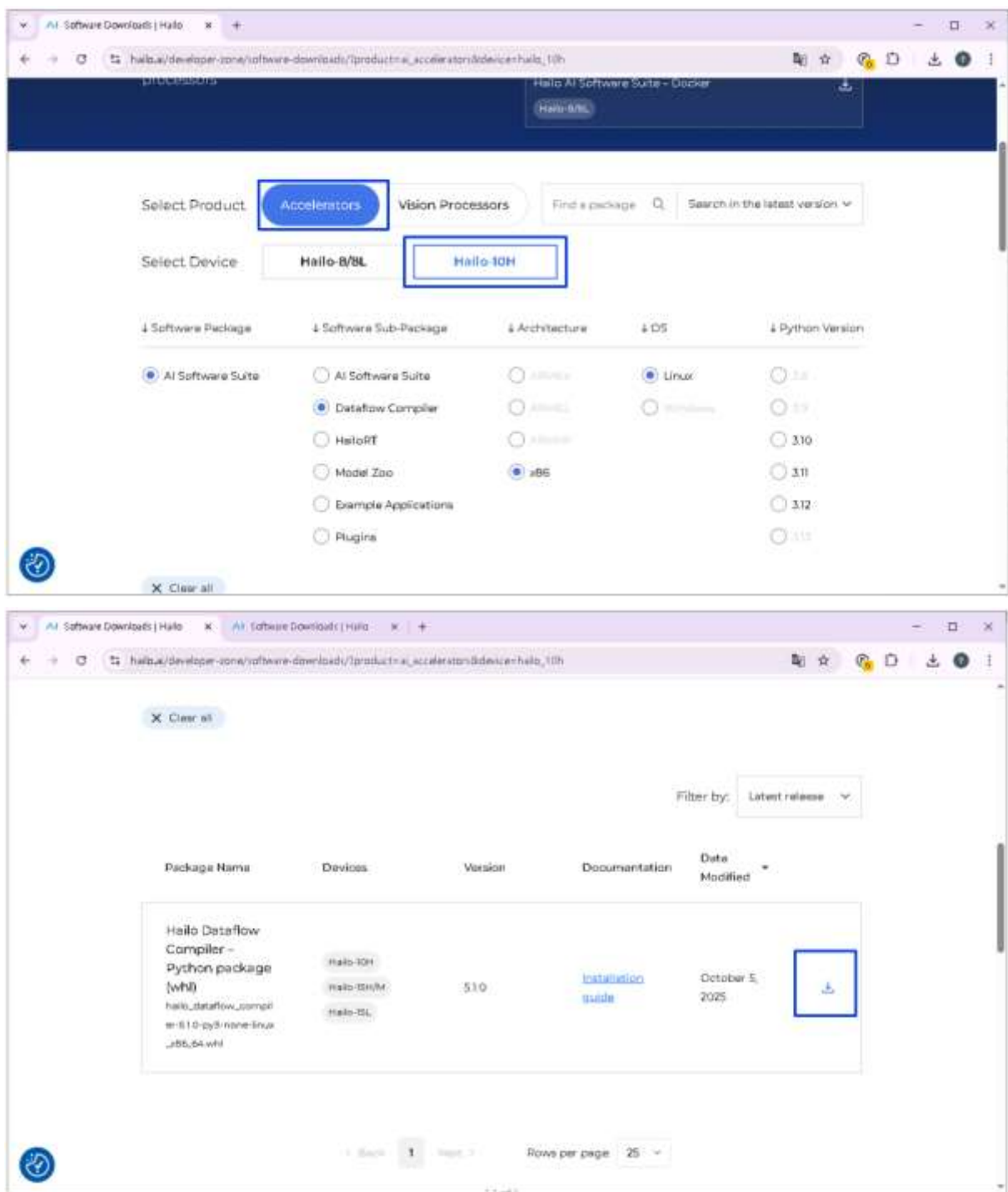
1. Visit <https://hailo.ai/developer-zone> , click the Sign up button, and proceed with the registration process.





The screenshot shows a web browser window with the URL `hailo.ai/developer-zone/request-access/`. The page has a dark background with the Hailo logo at the top left. The main heading is "Request access to explore Hailo's offerings". Below this, there is a form with four input fields: "Work Email*", "South Korea" (a dropdown menu), "First name*", and "Last name*". Below the form, there is a checkbox with the text "I agree to receive offers and promotions about our products and services". Below the checkbox, there is a line of text: "By providing your information above and clicking on the submit button, you agree to our [Terms of Use](#), [Privacy Policy](#) and [EULA](#)". At the bottom of the form is a large blue button labeled "SUBMIT" with a right-pointing arrow. In the bottom left corner of the page, there is a small circular icon with a stylized 'H' and a clock.

2. You can create an account immediately by entering the verification code sent to the email address you used when signing up.
3. Permissions must be granted to the created account to download the dfc for Hailo-15. Please contact Seyon Tech in advance to grant authorization .
4. After being granted permission, log in and go to the address below.
<https://hailo.ai/developer-zone/software-downloads/>
5. Select as shown below and download the `hailo_dataflow_compiler-5.1.0-py3-none-linux_x86_64.whl` file.



B. System Requirements

The minimum system requirements for installing the Hailo Dataflow Compiler are as follows.

Hardware and software requirements Required:

- Ubuntu 22.04 or 24.04 (x86_64), or Windows (via WSL2)
- RAM: 16GB or more (32GB recommended)
- Python 3.9, 3.10, or 3.12 (including pip and virtualenv)
- Required packages: python3.X-dev, python3-tk, graphviz, libgraphviz-dev

Additional GPU-based emulation requirements (optional):

- Nvidia GPU (Pascal/Turing/Ampere architecture)
- GPU driver version 555+
- CUDA 12.5.1 and CUDNN 9.10

C. Installing Dataflow Compiler

Install Dataflow Compiler using the downloaded whl file.

Preliminary Preparation – Install Essential Packages:

```
sudo apt-get update
sudo apt-get install python3-dev python3-tk graphviz libgraphviz-dev python3-venv
```

[Note] When using Ubuntu 24.04

– The python3-tk package may not be installed separately. Please install it using the command below:

```
sudo apt-get install python3-tk
```

– It runs on 16GB or more of RAM, and 32GB or more is recommended. – Model compilation in CPU mode is possible even without GPU/CUDA/CUDNN. – Even if the "System requirements check failed" message appears during installation verification, enter Y to continue.

1. Create a Python virtual environment:

```
python3 -m venv hailo_dfc_venv
```

2. Activate Virtual Environment:

```
source hailo_dfc_venv/bin/activate
```

3. Install Dataflow Compiler:

```
pip install hailo_dataflow_compiler-5.1.0-py3-none-linux_x86_64.whl
```

D. Verifying Installation and Introducing Key CLI Tools

Once the installation is complete, verify with the following command:

```
pip freeze | grep hailo  
hailo -h
```

Major CLI Tools:

- hailo parser: Convert ONNX/TFLite models to HAR format
- hailo optimize: Model quantization and optimization
- hailo compiler: generate HEF file
- hailo profiler: Model performance analysis
- hailo tutorial: Run Jupyter Notebook Tutorial

E. Deep Learning and ONNX File Generation

The AI SDK user must personally set up the dataset and the training environment for deep learning on a PC .

In Hailo, the training environment source code , pre-trained model files (ONNX), and compiled model files (HEF) tested on Hailo-15 are provided in the Hailo Model Zoo.

The link below contains information on representative AI models, and Developers can port this publicly available trained AI model to FlexWATCH® IP cameras for verification .

Classification:

https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/public_models/HAILO15H/HAILO15H_classification.rst

Object Detection:

https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/public_models/HAILO15H/HAILO15H_object_detection.rst

[ai/hailo_model_zoo/blob/master/docs/public_models/HAILO15H/HAILO15H_object_detection.rst](https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/public_models/HAILO15H/HAILO15H_object_detection.rst)

Segmentation:

https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/public_models/HAILO15H/HAILO15H_semantic_segmentation.rst

Other tasks:

https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/PUBLIC_MODELS.rst

F. Model Compilation Example (ONNX to HEF)

This section provides an example of the complete workflow for compiling an ONNX model into an HEF file for Hailo devices using Dataflow Compiler. The compilation process consists of the following three steps:

- 1) Parsing: Convert ONNX/TFLite models to Hailo Archive (HAR) format
- 2) Optimization: Model quantization (float32 → int8)
- 3) Compilation: Create HEF (Hailo Executable File)

[Step 1] ONNX Model Parsing

Converts ONNX models to Hailo Archive (HAR) format.

```
from hailo_sdk_client import ClientRunner #

Compile settings for
Hailo-15 devices chosen_hw_arch = "hailo15h" onnx_model_name = "yolov5s" onnx_path =
"./models/yolov5s.onnx"

# Initialize ClientRunner and parse ONNX model
runner = ClientRunner(hw_arch=chosen_hw_arch)hn, npz =
runner.translate_onnx_model(onnx_path, onnx_model_name)

# Save HAR file
```

```
runner.save_har(f"{onnx_model_name}_hailo_model.har")
```

[Step 2] Model Optimization (Optimization/Quantization)

Quantize the model using the calibration dataset. A minimum of 1,024 images is recommended.

```
import numpy as np
from hailo_sdk_client import ClientRunner

runner = ClientRunner(har="yolov5s_hailo_model.har")
calib_dataset = np.load("calib_set.npy")
runner.optimize(calib_dataset)
runner.save_har("yolov5s_quantized_model.har")
```

[Step 3] Compile HEF file

Compiles the quantized model into an executable HEF file on the Hailo device.

```
from hailo_sdk_client import ClientRunner

runner = ClientRunner(har="yolov5s_quantized_model.har")
hef = runner.compile()

with open("yolov5s.hef", "wb") as f:
    f.write(hef)
```

[Compile with CLI commands]

You can also execute each step using CLI commands instead of the Python API:

```
# 1. Model Parsing
hailo parser onnx yolov5s.onnx --hw-arch hailo15h

# 2. Model Optimization
hailo optimize yolov5s_hailo_model.har --calib-set-path calib_set.npy

# 3. HEF Compilation
hailo compiler yolov5s_quantized_model.har

# Performance Profiling
hailo profiler yolov5s_compiled_model.har
```

The generated HEF file can be uploaded to the Hailo-15 IP Camera and used in the Detection Demo.

4. Hailo-15 IP Camera Detection Demo Execution Guide

A. Overview

This section describes the procedure for running an object detection demo on a Hailo-15 IP Camera as an example of executing an HEF file compiled with Dataflow Compiler. It covers how to build and run a basic YOLOv5-based detection pipeline.

B. Prerequisites

Hardware : Hailo-15 IP Camera, Windows PC

Software :

- SSH client (e.g., PuTTY, Windows CMD/PowerShell)
- SCP clients (e.g., WinSCP, Windows CMD/PowerShell)
- RTP stream playback player (e.g., VLC Media Player)

Required files :

- HAILO-15_IP_Camera_detection.zip

C. Network Configuration

To run the demo, configure the network between the camera and the host PC as follows.

1. **IP Camera Network Settings** : Set the camera's IP address to 10.0.0.1 . (Subnet Mask: 255.255.255.0)

IP Address : Static IP

Static IP ☒ DHCP Client ☐

IP Address	10.0.0.1	
NetMask	255.255.255.0	
GateWay	10.0.0.254	
DNS 1	168.126.63.1	
DNS 2		

2. **Windows PC Network Settings** : Set the PC's Ethernet adapter IP address to 10.0.0.2 .
(Subnet Mask: 255.255.255.0)

5. Demo Environment Initialization

This section explains the process of running the trained model extracted using Dataflow Compiler on FlexWATCH® IP cameras. You can verify its operation in the GStreamer pipeline before final integration.

First, initialize the camera environment to prevent conflicts with existing running services.

A. Log in to the IP Camera using SSH. (password: root).

Log in to the IP Camera via SSH .

```
ssh root@10.0.0.1
```

B. Environment Preparation

To create an environment to run the demo sample, run the following command.

```
root@NetCAM:~# ./fwdaemonctl.sh disable
```

This command is the key point of this section. It is the core process of disabling the FlexWATCH® IP camera's video pipeline and running the previously prepared training model in the GStreamer video pipeline.

C. Reboot the IP Camera.

Reboot the IP Camera.

```
root@NetCAM:~# reboot
```

To run a natively built application, follow step 6 (Preparing configuration files required to run a native application) of 'How to Install SDK and Build Natively' in this manual starting from here.

D. Upload Demo Files.

Unzip the necessary files.


```
Unzip HAILO-15_IP_Camera_detection.zip
```

Once extraction is complete, you will see three files.

```
H 2 64.sdp
detection . zip
manage_config_tuning.sh
```

Once the IP camera boots up, please upload the necessary files to the IP camera .

```
scp -O .\manage_config_tuning.sh root@10.0.0.1:/home/root/apps
scp -O detection.zip root@10.0.0.1:/home/root/apps/detection/
```

E. Log in to IP Camera and prepare to run the demo.

To run the demo, please log in to the IP camera via SSH.

```
root@NetCAM:~/apps# chmod +x manage_config_tuning.sh
root@NetCAM:~/apps# cd detection/
root@NetCAM:~/apps/detection# ls detection.zip
resources
root@NetCAM:~/apps/detection# unzip detection.zip
Archive: detection.zip
warning: detection.zip appears to use backslashes as path separators
inflating: resources/yolov5m_wo_spp_60p_nv12_640.hef
inflating: resources/yolov5m_wo_spp_60p_nv12_fhd.hef
inflating: resources/yolov8s.hef
inflating: resources/configs/denoise_analytics_encoder_config.json
inflating: resources/configs/denoise_analytics_frontend_config.json
inflating: resources/configs/encoder_config.json
inflating: resources/configs/frontend_config.json
inflating: resources/configs/osd.json
inflating: resources/configs/osd_hailo_static_image.png
inflating: resources/configs/yolov5.json
inflating: resources/configs/yolov5_personface.json
inflating: denoise_analytics.sh
inflating: detection.sh
```

F. Run /home/root/apps/detection/detection.sh.

Run the detection.sh script.

```

root@NetCAM:~/apps/detection# chmod +x detection.sh
denoise_analytics.sh          detection.sh          detection.zip
resourcesroot@NetCAM:~/apps/detection# ./detection.shStarting config tuning
application with mode: daylightConfig tuning application started with PID: 2813Stdout
and Stderr log output: /tmp/config_tuning_app.log, medialib log output:
/tmp/config_tuning_app/medialib.logUse '/home/root/apps/manage_config_tuning.sh
stop' to stop the applicationUse '/home/root/apps/manage_config_tuning.sh status' to
check statusRunning yolov5 with MODE=daylightgst-launch-1.0 hailofrontendbinsrc
config-file-path=/home/root/apps/detection/resources/configs/frontend_config.json
name=frontend frontend. ! queue leaky=no max-size-buffers=5 max-size-bytes=0 max-
size-time=0 ! hailonet hef-
path=/home/root/apps/detection/resources/yolov5m_wo_spp_60p_nv12_fhd.hef
scheduling-algorithm=1 vdevice-group-id=device0 ! queue leaky=no max-size-
buffers=5 max-size-bytes=0 max-size-time=0 ! hailofilter function-name=yolov5
config-path=/home/root/apps/detection/resources/configs/yolov5.json so-
path=/usr/lib/hailo-post-processes/libyolo_post.so qos=false ! queue leaky=no max-
size-buffers=5 max-size-bytes=0 max-size-time=0 ! hailooverlay qos=false ! queue
leaky=no max-size-buffers=5 max-size-bytes=0 max-size-time=0 ! hailoencodebin
config-file-path=/home/root/apps/detection/resources/configs/encoder_config.json !
h264parse config-interval=-1 ! video/x-h264,framerate=30/1 ! tee name=udp_tee
udp_tee. ! queue leaky=no max-size-buffers=5 max-size-bytes=0 max-size-time=0 !
rtph264pay! 'application/x-rtp, media=(string)video, encoding-name=(string)H264' !
udpsink host=10.0.0.2 port=5000 name=udp_sink sync=false udp_tee. ! queue
leaky=no max-size-buffers=5 max-size-bytes=0 max-size-time=0 ! fpsdisplaysink fps-
update-interval=2000 video-sink=fakesink name=hailo_display sync=false text-
overlay=false

(gst-plugin-scanner:2815): GStreamer-WARNING **: 11:04:35.833: Failed to load
plugin '/usr/lib/gstreamer-1.0/libgstinstrumentso.so': /usr/lib/gstreamer-
1.0/libgstinstrumentso.so: undefined symbol: for_each_bufferSetting pipeline to
PAUSED ...Pipeline is live and does not need PREROLL ...Pipeline is PREROLLED ...Setting
pipeline to PLAYING ...
New clock: GstSystemClock
Redistribute latency...Redistribute latency...Redistribute latency...Redistribute
latency...0:00:57.0 / 99:99:99.

```

G. View execution results.

If you double-click the h264.sdp file (open with VLC) on a Windows PC or open the H264.sdp file in VLC, you can check the execution results of the detection.sh file as shown below. Small objects are detected well. In other words, you can confirm that the

AI model uploaded from the IP camera is working.

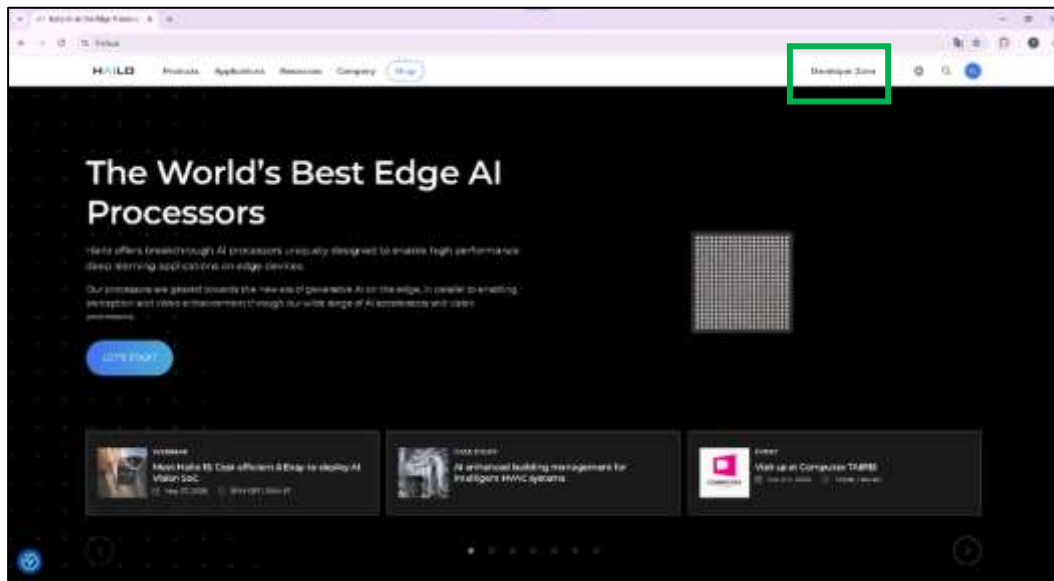


the shell script file and HEF (Hailo Executable Format) file in the manner described above and perform inference using your own AI model.

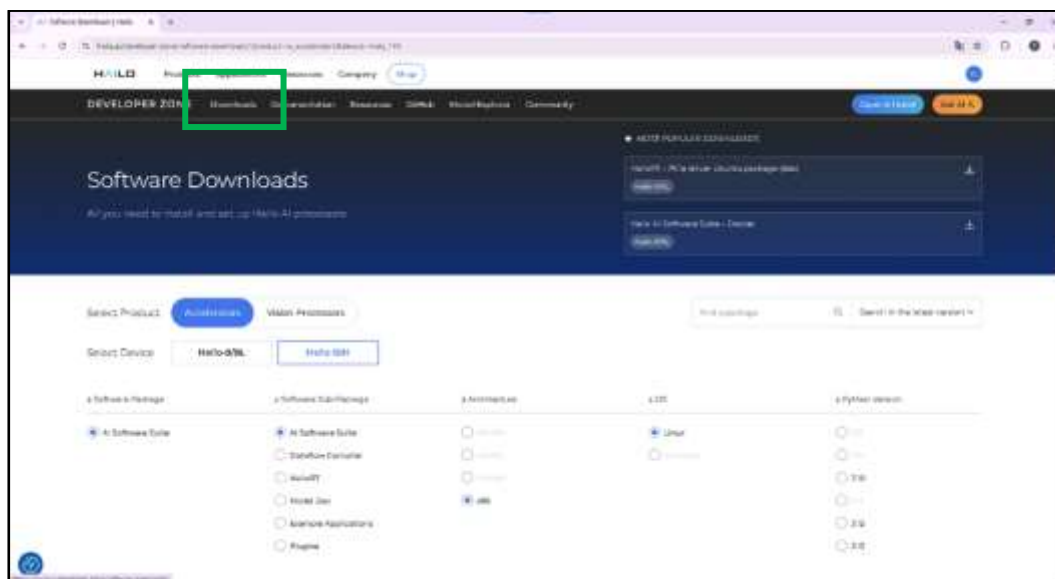
6.How to Install SDK and Build Natively

native build, download the necessary software packages from the Hailo Developer Zone by following the procedure below:

1. Go to <https://hailo.ai> and click 'Developer Zone'.

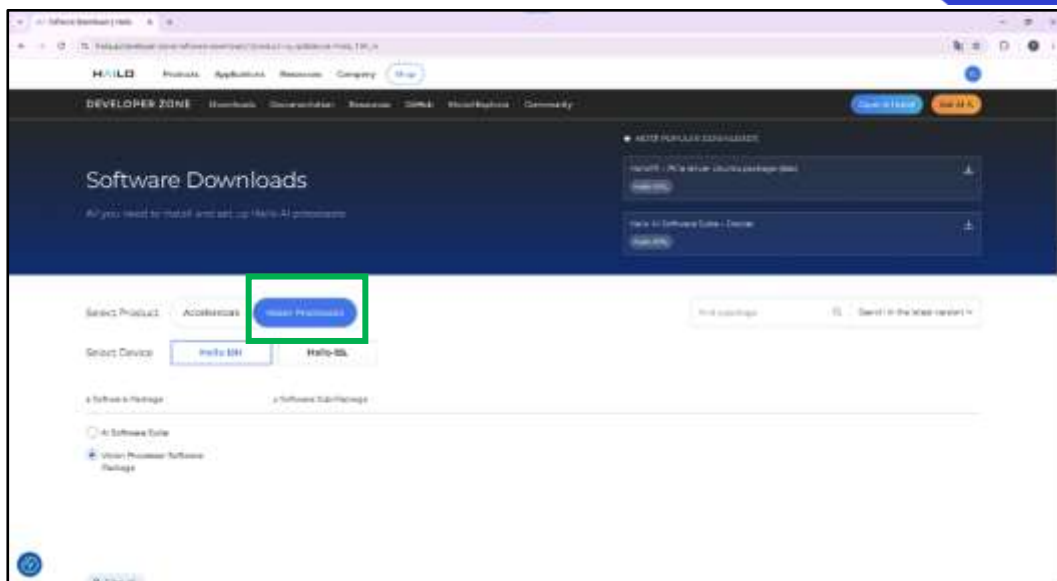


2. Go to the 'Downloads' menu.



3. Navigate to the 'Software Downloads' menu.

4. Select 'Vision Processors' from the Product category.



5. Locate and download the 'Hailo-15H Vision Processor Software Package SBC 2.X' file.



This document describes the process of installing SDK 1.11.0 for the Hailo-15H and building the corresponding Hailo Media Library v1.11.0 source code. Please copy and use all commands in order.

1. Install SDK 1.11.0 (Host PC)

If it is already installed, you can skip this step, but if you want to install it or check the path, use the command below.

```
sudo sh /home/manager/Desktop/prebuilt/sbc/sdk/poky-glibc-x86_64-core-image-  
minimal-armv8a-hailo15-sbc-toolchain-4.0.23.sh -y -d /opt/poky/4.0.23
```

2. Hailo Media Library v1.11.0 Source Clone

```
cd /home/manager/github-repository  
  
git clone https://github.com/hailo-ai/hailo-media-library.git hailo-media-library-  
1.11.0  
  
cd hailo-media-library-1.11.0  
git checkout v1.11.0
```

3. Build Environment Setup and Compilation

3.1. Loading SDK Environment Variables

You must run it once per terminal session.

```
. /opt/poky/4.0.23/environment-setup-armv8a-poky-linux
```

3.2. Meson Build Configuration

Create a build directory including the Hailo-15H platform configuration.

```
meson build
```

3.3. Start Ninja Compilation

Generates the actual library and application binary.

```
ninja -C build
```

4. Check the build results

Once the build is complete, you can check the following files in the `build` folder.

- Example app: `build/hailo-analytics/apps/face\_landmarks/face\_landmarks\_app`

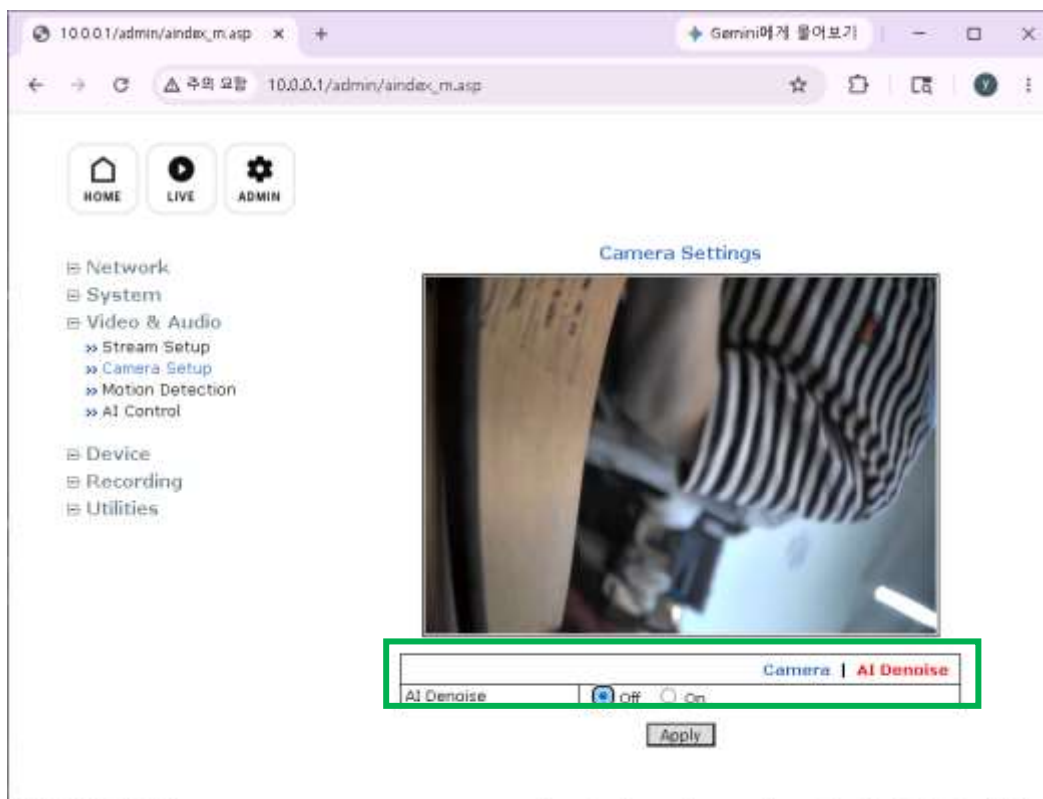
5. Send to Device (Hailo-15)

Use SCP to transfer the built file to the device .

```
scp face_landmarks_app root@10.0.0.1:/root/  
s cp lpr_app root@10.0.0.1:/root
```

6. Turn off AI Denoise mode

Admin -> Video & Audio -> Camera Setup Set AI Denoise to off in the menu and press the Apply button.



7. Preparing configuration files required to run the native application

In the downloaded FlexWatch_Hailo-15_AI_SDK_1.0_20260520, navigate as follows.

SDK > FlexWatch_Hailo-15_AI_SDK_0.2_20260520 > sdk_1.11.0 > H15_AI_SDK.json				H15_AI_SDK
이름	수정한 날짜	유형	크기	
00_ai_example_app_original	2026-05-22 오후 3:19	파일 폴더		
deploy.bat	2026-05-22 오후 3:01	Windows 배치 파일	9KB	
libhailo_analytics.so.1.11.0	2026-05-22 오후 1:35	0 파일	101,272KB	
path.txt	2026-05-22 오후 2:00	텍스트 문서	1KB	

After connecting to the camera via SSH, execute the following command.

```
root@NetCAM:~# ./fwdaemonctl.sh disable
root@NetCAM:~# reboot
```

the camera reboot is complete, double -click the deploy.bat file on your Windows PC to run it. Execute the deploy.bat file as shown below. (Select bold font.)

, select 2 below to proceed.

[Caution] Profile 2 runs the camera's default firmware streaming service (fwdaemon), so it monopolizes NPU resources. As a result, the standalone native app (face_landmarks_app) built earlier will not work due to resource conflicts. When running and verifying the native app independently, you must select Profile 1 or keep fwdaemonctl.sh disabled.

```
(base) PS E:\H15_AI_SDK_json> .\deploy.bat
=====
H15 AI SDK Deploy & Restore Tool
=====
Select operation mode:
1) Deploy AI SDK Configuration
2) Restore Flexwatch Camera Configuration
Enter mode number (1 or 2): 1
Enter target device IP address: 10.0.0.1
Enter SSH username (default: root): root
Enter SSH port (default: 22): 22

=====
[Deployment Info Summary]
- Source Profile Folder: E:\H15_AI_SDK_json\00_ai_example_app_original
- Local Library File: E:\H15_AI_SDK_json\libhailo_analytics.so.1.11.0
- Target IP: 10.0.0.1
- Username: root
- Port: 22
-
      Target                                Profile                                Path:
/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/profiles/daylight/ai_example_dayli
ght/
- Target Library Path: /usr/lib/
=====

Do you want to proceed with deployment? (Y/N): Y

Fetching current timestamp for backup...
Backup Timestamp: 20260522_144831
```



```
=====
[STEP 1/5] Backing up existing target files & folders...
=====
```

```
root@10.0.0.1's password:
```

```
Profile backed up successfully.
```

```
Library backed up successfully.
```

```
=====
[STEP 2/5] Deploying profile files...
=====
```

```
root@10.0.0.1's password:
```

```
Permission denied, please try again.
```

```
root@10.0.0.1's password:
```

```
ai_example_daylight_profile.json 100% 1642 534.5KB/s 00:00
```

```
application_settings.json 100% 1697 552.4KB/s 00:00
```

```
calib.json 100% 79KB 7.0MB/s 00:00
```

```
encoder_sink0.json 100% 2594 506.6KB/s 00:00
```

```
encoder_sink1.json 100% 2592 843.7KB/s 00:00
```

```
iq_settings.json 100% 17KB 3.3MB/s 00:00
```

```
masking_sink0.json 100% 361 117.5KB/s 00:00
```

```
masking_sink1.json 100% 361 176.3KB/s 00:00
```

```
osd_sink0.json 100% 320 104.2KB/s 00:00
```

```
osd_sink1.json 100% 320 104.2KB/s 00:00
```

```
sensor_config.json 100% 717 233.4KB/s 00:00
```

```
stabilizer_settings.json 100% 1509 491.2KB/s 00:00
```

```
[SUCCESS] Profile files copied successfully.
```

```
=====
[STEP 3/5] Deploying library file...
=====
```

```
root@10.0.0.1's password:
```

```
libhailo_analytics.so.1.11.0 100% 99MB 10.9MB/s 00:09
```

```
[SUCCESS] Library file copied successfully.
```

```
=====
[STEP 4/5] Rebooting target device...
=====
```

```
root@10.0.0.1's password:
```

```
[SUCCESS] Reboot command sent. Device is restarting...
```

```
=====
[SUCCESS] Deployment flow complete.
=====
```

Press any key to continue. . .

Reboot the camera.

8. Run on Device (Hailo-15)

Run the app on the device.

face_landmarks_app

```
./face_landmarks_app -o 10.0.0.2 -t 999999999
```

lpr _ app

```
./lpr_app -o 10.0.0.2 -t 999999999
```

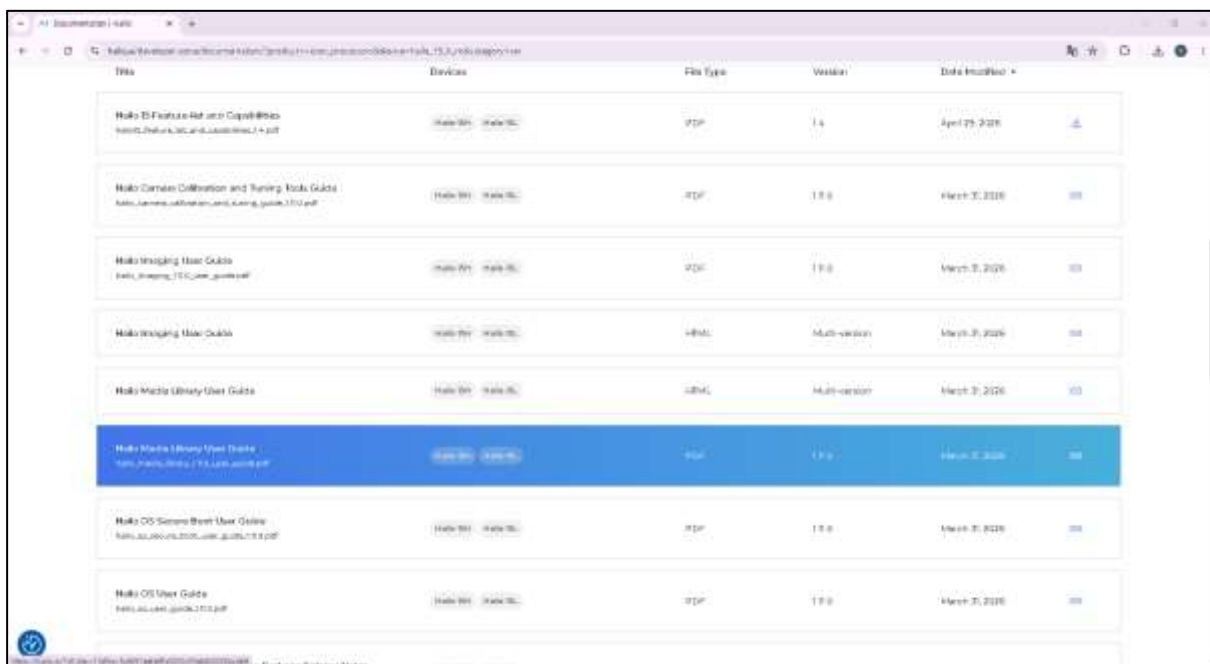
Enter the IP address of the host PC to check the video (10.0.0.2 set in the previous step) using the -o option.

9. Host PC Check video at

Hailo.ai Download hailo_media_library_1.11.0_user_guide.pdf from the site .

Documentation -> Vision Processors -> hailo_media_library_1.11.0_user_guide.pdf

Download



hailo_media_library_1.11.0_user_guide.pdf Refer to the next page of the document for the app

Execute.

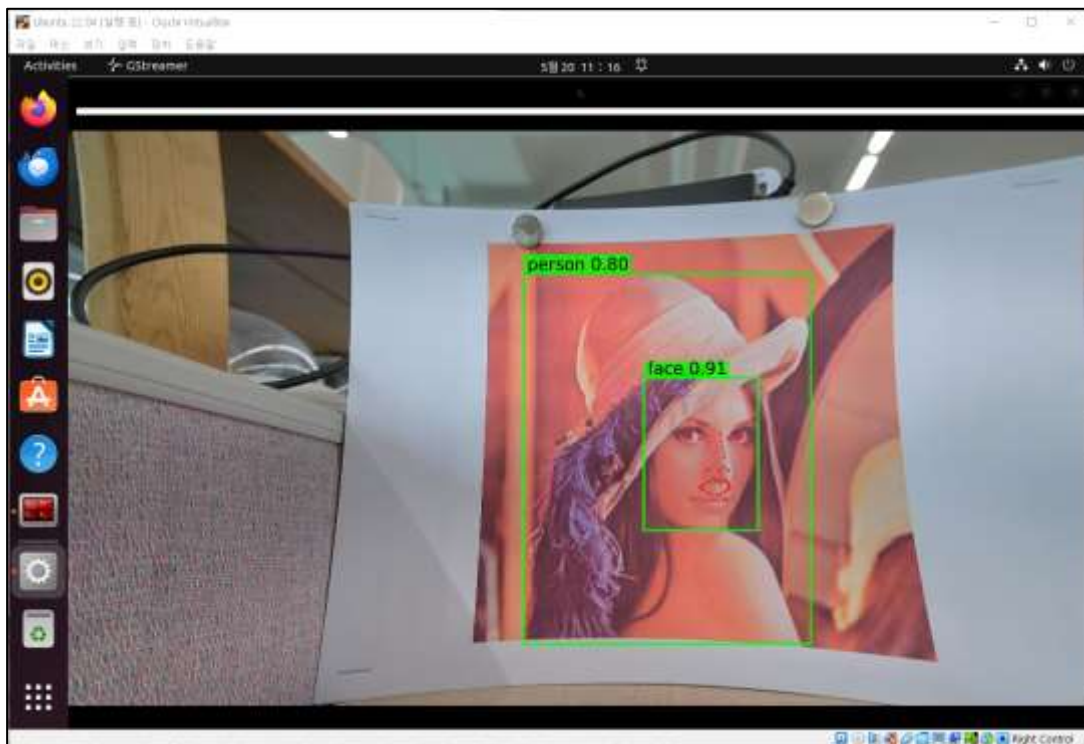
Host PC Environment Setup & Virtual Environment Configuration: Page 343

Analytic Viewer App Launch Guide: Page 351

If the video does not play, you need to check the firewall settings on the host PC.

You can view the execution screen of the face_landmark app as shown in the image below .

<Face_landmark_ Viewer Execution Screen>



< Lpr Viewer Execution Screen >



You can also view the video using the .sdp file below .

With the content below If you create the stream_vlc.sdp file and drag and drop it into VLC

Player, the video will play.

```
v=0
o=- 0 0 IN IP4 127.0.0.1
s=GStreamer RTP Stream
c=IN IP4 < Host PC IP Address>
t=0 0
m=video 5000 RTP/AVP 96
a=rtpmap:96 H264/90000
```

7. Integration of .sh, HEF files on FlexWATCH® IP-Camera.

Once it is confirmed that the AI model developed by the customer operates correctly on the GStreamer-based Video Pipeline of the FlexWATCH® IP camera through the process described in the previous section, the step of finally integrating the model into the **FlexWATCH® Video Pipeline** is carried out.

Currently, this integration process is performed directly by Seyon Tech and provided as individual support for each client. This is because the structure and execution methods of AI models differ for each client, and the forms of metadata generated from AI inference results also vary. Furthermore, since the Hailo platform supports greater diversity compared to others, collecting a significant amount of information is required to standardize the integration process. Under these circumstances, we have determined that individual integration support is the most realistic approach. As a matter of principle, we provide integration support free of charge, unless separate source code-level development is required.

Under current policy, upon completion of AI model development, the client company delivers the **Hailo Execution File (HEF)** and the **Script file (.sh)** for execution to Seyon Tech. Based on this, Seyon Tech performs the task of integrating the client's AI model into the Video Pipeline to ensure it operates correctly in a FlexWATCH® IP camera environment. This process requires consideration of the camera's internal structure and system dependencies, and is currently conducted under the responsibility of Seyon Tech. Occasionally, some clients express concerns regarding the external leakage of the HEF file. In such cases, they may provide a dummy HEF file and Script file with identical Input/Output Shapes to allow us to verify the structural integration of the GStreamer pipeline; alternatively, we recommend providing an incomplete version of the HEF file and allowing the client to manage the production version themselves via the web in the future.

its own AI IP cameras supporting various standard protocols such as ONVIF, RTP/RTSP, HTTP, and HTTPS. Subsequently, HEF files can be uploaded or replaced via the camera's web configuration interface, and the client can directly manage the upgrade and operation of the AI model.

Additionally, a guide on license key-based management methods will be provided to prepare for situations where security protection of the AI model is required.

to provide a Standard AI SDK by maintaining the AI SDK through these individual integration support policies for a certain period (e.g., about one year) , and then gradually standardizing the integration process based on accumulated cases and technical experience . Ultimately, the goal is to establish an environment where client companies can independently port and integrate AI models using only the SDK documentation, without the need for separate integration service support.

Based on this direction, the FlexWATCH® AI SDK will be continuously expanded and enhanced, and will evolve into an open IP camera platform capable of flexibly accommodating various AI models from client companies .

8. Appendix

GStreamer Pipeline Analysis

7.1.1 GStreamer Basic Concepts

GStreamer is a multimedia framework composed of the following core concepts.

concept	explanation
Element	Functional units (camera, encoder, decoder, etc.)
Pad	Connection points between elements (source pad → sink pad)
Pipeline	Connection structure of elements

7.1.2 Pipeline Element Details

This is a detailed description of each Element used in the Detection demo pipeline.

7.1.2.1 hailofrontendbinsrc (source)

item	detail
Element name	hailofrontendbinsrc
role	Hailo Camera Interface ISP (Image Signal Processor) control

item	detail
	Sensor settings (resolution, FPS, format)
parameters	config-file-path : Camera settings JSON file path
example	hailofrontendbinsrc config-file-path=/home/root/apps/detection/resources/configs/frontend_config.json

7.1.2.2 queue (buffer)

item	detail
Element name	queue
role	Ensuring pipeline stability through data buffering Controlling data flow between elements
parameters	leaky : Action on buffer overflow (no: wait, upstream: drop previous data, downstream: drop latest data) max-size-buffers : Maximum number of buffers max-size-bytes : maximum buffer size (bytes) max-size-time : Maximum buffer time (nanoseconds)
example	queue leaky=no max-size-buffers=5 max-size-bytes=0 max-size-time=0

7.1.2.3 hailonet (AI Inference)

item	detail
Element name	hailonet
role	Performing deep learning model inference on Hailo NPU Load and run HEF file
parameters	hef-path : Path to model files (.hef) compiled for the Hailo NPU scheduling-algorithm : Scheduling algorithm (0: default, 1: priority-based) vdevice-group-id : NPU device group ID to use
example	hailonet hef-path=/home/root/apps/detection/resources/yolov5m_wo_spp_60p_nv12_fhd.hef scheduling-algorithm=1 vdevice-group-id=device0

7.1.2.4 hailofilter (post-processing)

item	detail
Element name	hailofilter
role	Convert NPU output into actual object information Apply NMS (Non-Maximum Suppression) Confidence threshold filtering
parameters	function-name : Post-processing function name (yolov5, ssd, etc.)

item	detail
	so-path : Post-processing library path config-path : Path to post-processing configuration JSON file qos : Whether QoS (Quality of Service) is enabled
example	hailofilter function-name=yolov5 config-path=/home/root/apps/detection/resources/configs/yolov5.json so-path=/usr/lib/hailo-post-processes/libyolo_post.so qos=false

7.1.2.5 hailooverlay (visualization)

item	detail
Element name	hailooverlay
role	Drawing a bounding box Class name text rendering Display Confidence score
parameters	qos : Whether QoS (Quality of Service) is enabled
example	hailooverlay qos=false

7.1.2.6 hailoencodebin (encoder)

item	detail
Element name	hailoencodebin
role	Hailo hardware encoder wrapper Perform H.264/H.265 encoding
parameters	config-file-path : Path to encoder configuration JSON file
example	hailoencodebin config-file-path=/home/root/apps/detection/resources/configs/encoder_config.json

7.1.2.7 tee (branch)

item	detail
Element name	tee
role	Duplicate the stream across multiple paths Distribute a single input into multiple outputs
parameters	name : Element name (for reference by other elements)
example	tee name=udp_tee

7.1.3 Pipeline Configuration Example

The complete pipeline configuration used in the Detection demo is as follows:

Complete Configuration of the Detection Pipeline

```
gst-launch-1.0 \  
hailofrontendbinsrc config-file-path=frontend_config.json name=frontend \  
frontend. ! queue leaky=no max-size-buffers=5 ! \  
hailonet hef-path=yolov5m.hef scheduling-algorithm=1 ! \  
queue! hailofilter function-name=yolov5 so-path=libyolo_post.so ! \  
queue! hailooverlay qos=false ! \  
queue! hailoencodebin config-file-path=encoder_config.json ! \  
h264parse config-interval=-1 ! tee name=udp_tee \  
udp_tee. ! queue! rtph264pay! udpsink host=10.0.0.2 port=5000 \  
udp_tee. ! queue! fpsdisplaysink video-sink=fakesink
```

Pipeline data flow

Camera → Preprocessing → NPU Inference → Postprocessing → Visualization → Encoding →
Branching
├→ UDP transmission
└→ FPS Monitoring

Troubleshooting when viewing RTSP video with VLC

If you are unable to play RTSP streams in VLC Media Player, you can resolve the issue by changing the network caching options as follows.

Setup method:

Launch VLC Media Player.

Select [Tools] → [Preferences] from the menu.

Change "Show settings" at the bottom left to "All".

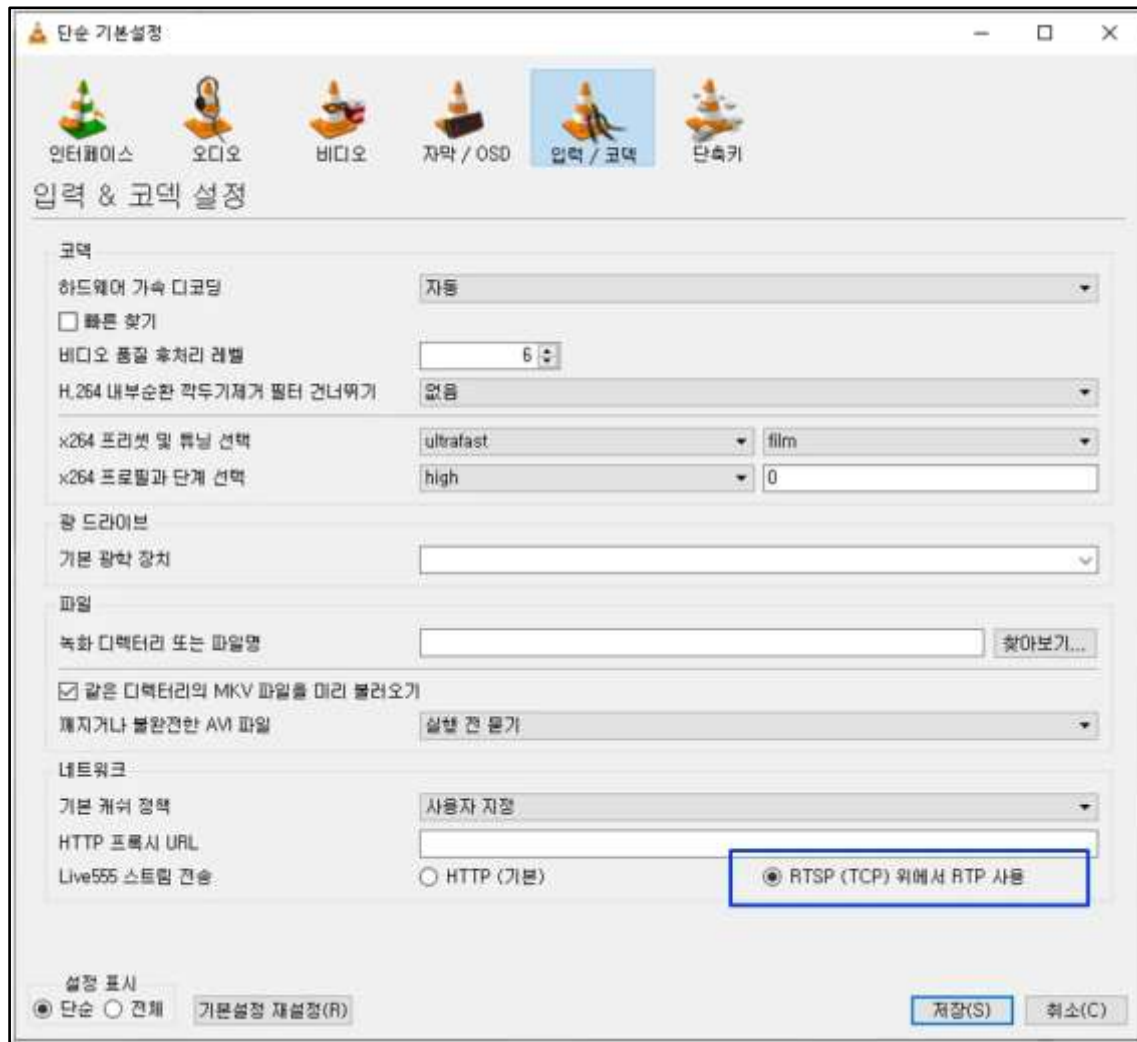
Select [Input / Codec] → [Input & Codec Settings] from the tree on the left.

In the "Live555 stream transmission" section, select the "Use RTP over RTSP (TCP)" option.

Click the [Save] button to save the settings.

After restarting VLC, play the RTSP stream again.

Note: This setting improves network stability by transmitting RTSP streams over the TCP protocol. It is particularly useful in environments where packet loss occurs with the UDP method.



[Image] VLC Preferences - Use RTP over RTSP (TCP) option

1. Go to <https://hailo.ai> and click 'Developer Zone'.